Prompt Engineering Lab

Structured AI-Assisted Codegen with Spring Boot — Prompt Chain & Pattern Guide

Version 2.0 | Last updated: July 14, 2026 | Audience: students with working Java/Spring Boot/REST/Maven knowledge | Format: reference document, not yet validated end-to-end against a live model

---

1. Overview

This document is a prompt-engineering lab for generating a Spring Boot "User Registration Service" through a fully AI-assisted, phase-locked, level-gated workflow. It starts with the general methodology and pattern taxonomy behind the lab, then pairs each prompt with the named pattern it demonstrates, so students understand not just what to paste but which reusable technique is at work, which category it belongs to, and why it's structured that way.

Sections:

- Section 2 — Prompt engineering methodology: the pattern taxonomy and the prompt lifecycle
- Section 3 — Definitions of every pattern in the taxonomy, marking which are used in this lab's prompt chain
- Section 4 — The full prompt chain, ready to paste into a chat session
- Section 5 — Notes for students

2. Prompt Engineering Methodology

Before looking at any single prompt, it helps to see the bigger picture: prompt-engineering patterns fall into a small number of categories, and a well-formed AI-assisted workflow moves through a predictable lifecycle from clarification to delivery. This lab is one concrete implementation of that lifecycle.

2.1 Pattern Taxonomy

Every named pattern used in prompt engineering falls into one of six categories, based on what problem it solves rather than what it looks like on the page.

Pattern Category	Prompt Pattern
Input Semantics	*Meta Language Creation*
Output Customization	*Output Automater* *Persona* *Visualization Generator* *Recipe* *Template*
Error Identification	*Fact Check List* *Reflection*
Prompt Improvement	*Question Refinement* *Alternative Approaches* *Cognitive Verifier* *Refusal Breaker*
Interaction	*Flipped Interaction*

Pattern Category	Prompt Pattern
	Game Play *Infinite Generation*
Context Control	*Context Manager*

Patterns marked with an asterisk (*) in Section 3 are the ones actually used in this lab's prompt chain; the taxonomy above is broader and includes patterns useful in other projects.

2.2 The Prompt Lifecycle (v1.0)

A mature prompt-engineering workflow is not a single prompt — it is a sequence of stages, several of which loop. The stages below map directly onto the phases used later in this lab.

Stage 0 — Clarification (iterative)

Clarify vague, ambiguous, or incomplete requests. This can happen at any point in the conversation, not just at the start.

- Pattern: Question Refinement

↓

Stage 1 — System Prompt

Define the AI's behavior, scope, persona, rules, and boundaries before any task-specific work begins.

- Pattern: Context Manager

↓

Stage 2 — User Prompt

The task-specific prompt itself, built from two layers: its components and the construction patterns used to assemble them.

Prompt components:

- Role
- Context
- Task
- Requirements
- Constraints
- Examples
- Output format

Construction patterns:

- Persona
- Meta Language Creation
- Recipe Pattern
- Template Pattern
- Visualization Generator
- Output Automator

↓

Stage 3 — Reasoning Strategy

Choose how the model should think through the task before it answers.

Single-pass reasoning:

- Step-Back
- Chain of Thought (when appropriate)
- Structured Reasoning

Multi-sample strategies:

- Tree of Thoughts
- Self-Consistency

Agentic:

- ReAct

↓

Stage 4 — Response Generation

Generate the initial response using the prompt and the selected reasoning strategy.

↓

Stage 5 — Response Enhancement

Improve the raw response before it's treated as a candidate answer.

Verification patterns:

- Reflection
- Fact Check List
- Cognitive Verifier

Domain-specific prompting:

- Code Generation
- Refactoring
- Debugging
- Testing
- Documentation
- Technical Writing
- Data Analysis
- Image Generation

Decision point — Requirements satisfied?

- YES → continue to Stage 6
- NO → refine the user prompt, switch reasoning strategy, or improve domain-specific instructions, then loop back to Stage 4

↓

Stage 6 — Response Evaluation & Optimization

Compare candidate responses, select the best one, and optimize it before delivery.

- Alternative Solutions
- Evaluation
- Selection
- Optimization

↓

Stage 7 — Output & Delivery

Deliver the final response in the format the task actually needs.

- Markdown
- JSON
- Mermaid
- Source Code
- Tables
- PDF

2.3 Cross-Cutting Interaction Patterns

Three patterns don't belong to a single stage — they can be applied at any point in the lifecycle above, and this lab uses all three:

- Flipped Interaction
- Game Play
- Infinite Generation

2.4 How This Lab Maps to the Lifecycle

Lifecycle Stage	Where it appears in this lab
0. Clarification	Prompt 5 — Question Refinement
1. System Prompt	Prompts 1–4 — Execution Policy, Personas, Context Manager
2. User Prompt	Prompts 6–9 — Meta Language, Few-shot, Output Template/Automator
3. Reasoning Strategy	Prompt 10 — Reasoning Scope (structured reasoning, trace suppressed)
4–5. Generation & Enhancement	Prompts 11–12 — Fact Check List (pre) and Reflection (post), applied per level
6. Evaluation & Optimization	Prompt 13 — Alternative Approaches, run on request
7. Output & Delivery	Levels 1–11 — source code, generated one level at a time
Cross-cutting	Prompts 14–16 — Flipped Interaction, Level Plan (Recipe + Game Play), Continuation Gate (Infinite Generation)

3. Prompt Engineering Patterns Used

This section defines every pattern named in the Section 2 taxonomy. Patterns marked with an asterisk (*) are used directly in this lab's prompt chain (Section 4); patterns without an asterisk are reference definitions — not used here, but useful to recognize when extending this lab or building others.

Input Semantics

3.1 Meta Language / Domain Glossary Pattern *

Defines project-specific shorthand once (e.g. “Entity = JPA Entity”) so every later prompt can use the short form without re-explaining it. Reduces token usage and keeps terminology consistent across a long session. Used in Prompt 6.

Output Customization

3.2 Persona Pattern (System + User) *

Assigns the model a role (“Principal Java Architect”) and assigns the human a role (“Senior Java Developer”). This calibrates vocabulary, depth, and the assumed baseline knowledge on both sides, avoiding beginner-level explanations the user doesn't need. Used in Prompts 2–3.

3.3 Template Pattern *

Fixes a consistent output structure — a set of named sections every response must fill in (Purpose, Design Decisions, Files Generated, etc.) — so output is predictable, scannable, and diffable across a long session. Used in Prompt 8.

3.4 Output Automator Pattern *

A standing instruction to automatically attach a fixed set of summary elements (Architecture Summary, Files Generated, Design Decisions, Next Logical Task) to every response, without being asked each time. Closely related to the Template pattern but distinct: Template defines the shape of a response, Output Automator guarantees it's applied without a fresh reminder. Used in Prompt 9.

3.5 Recipe Pattern *

Gives the model a fixed, ordered procedure to follow for a repeatable class of task — a checklist of steps in sequence, the way a cooking recipe fixes both the ingredients and the order of operations. In this lab, the Level Plan (Prompt 15) is a Recipe: eleven ordered steps, each with defined inputs and outputs.

3.6 Visualization Generator Pattern

Asks the model to represent a concept, structure, or flow visually — for example, as a Mermaid diagram, an ER diagram, or an ASCII tree — rather than describing it only in prose. Not used in this lab's chain, since the deliverable is source code rather than diagrams, but a natural extension: for example, prompting for a Mermaid ER diagram of the User entity, or a sequence diagram of the registration flow once JWT and Kafka are introduced.

Error Identification

3.7 Fact Check List Pattern *

Gives the model a fixed list of criteria to verify a piece of output against before treating it as final — a checklist applied as a gate, not a suggestion. Used in Prompt 11 (“Fast Checklist”), run before each level is generated.

3.8 Reflection Pattern *

After producing output, asks the model to critically re-examine its own work against a set of concerns (security, performance, layering, duplication) and report back, rather than treating the first draft as final. Used in Prompt 12, run after each level is generated — this is what closes the loop that Prompt 11 opens: check before, reflect after.

Prompt Improvement

3.9 Question Refinement Pattern *

Asks the model to silently improve the clarity and precision of each incoming instruction before acting on it, while preserving the original intent. Useful for catching ambiguous requirements early. Used in Prompt 5.

3.10 Alternative Approaches Pattern *

Prompts the model to compare several valid options (e.g. Layered vs. Hexagonal vs. Clean Architecture) and justify a recommendation, instead of committing to the first approach it considers. Best scoped to fire only on explicit request, so it doesn't add noise to routine, already-decided tasks. Used in Prompt 13.

3.11 Cognitive Verifier Pattern

When faced with a complex or underspecified question, the model generates the additional clarifying sub-questions it would need answered to give an accurate response, answers them, and combines the results into a final answer — verification built into how the question is decomposed, not applied afterward. Distinct from Fact Check List and Reflection, which verify output that already exists. Not used directly in this lab's chain, since Question Refinement (Prompt 5) already narrows ambiguity before generation; Cognitive Verifier is more useful for open-ended architecture or requirements questions than for the well-specified levels in this lab.

3.12 Refusal Breaker Pattern

Used when a request is declined for an incidental reason — ambiguous scope, missing context, an overly broad ask — by asking the model to explain what specifically blocked it and to suggest a narrower or better-specified version that it could act on. Not used in this lab's chain. Worth a caution: this pattern is for resolving legitimate ambiguity, not for working around a model's safety or policy decisions; if a request is declined on safety grounds, rephrasing to get around that decision is a misuse of the pattern, not an application of it.

Interaction

3.13 Flipped Interaction Pattern *

Tells the model to proceed on sensible assumptions rather than pausing to ask the user questions, except when a genuine blocker exists. Keeps a long build session moving. Used in Prompt 14.

3.14 Game Play Pattern *

Frames the task as a sequence of levels that unlock one at a time, gating progress on explicit confirmation rather than generating everything at once. In this lab, Game Play is the unlock mechanic layered on top of the Recipe pattern's fixed procedure (3.5) — Recipe defines what the eleven steps are, Game Play defines that they're gated one at a time. Used in Prompt 15.

3.15 Infinite Generation Pattern *

After each unit of work, the model offers the next logical step and waits for confirmation, without regenerating or duplicating previous output. This should be a single confirmation point per level — combined carelessly with Game Play's own "wait for confirmation" instruction, it's easy to end up with two redundant stop points per level instead of one. Used in Prompt 16.

Context Control

3.16 Context Manager Pattern *

Establishes durable facts about the project (tech stack, versions, architecture style) that should be treated as constraints for every subsequent prompt, rather than being restated each time. Used in Prompt 4, and again structurally in Prompt 1 to lock the session into a configuration phase before any implementation begins.

Patterns Used in This Lab, Outside the Six-Category Taxonomy

Two patterns used in this lab's prompt chain don't map cleanly onto the six categories in Section 2.1 — they're included here because they're active in Section 4, not because they belong to Input Semantics through Context Control.

3.17 Few-Shot Pattern *

Gives input/output examples ("Generate Repository → Repository only") so the model infers the desired scope and format by example rather than by explicit rule alone. Used in Prompt 7.

3.18 Reasoning Scope / Chain-of-Thought Suppression Pattern *

Asks the model to reason internally but present only the final answer plus a brief rationale — not a full step-by-step trace. The scope matters: suppressing reasoning entirely would starve the Template pattern's "Design Decisions" section (3.3), so this pattern should only ever suppress the step-by-step trace, never the final rationale. Used in Prompt 10; this pattern governs Stage 3 (Reasoning Strategy) of the lifecycle rather than input, output, error-checking, refinement, interaction, or context.

Sources and further reading: the pattern taxonomy and definitions in this section draw on the published prompt-pattern literature for large language models, notably White et al., "A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT" (2023), and later community extensions of that catalog. Names and groupings here have been adapted for this lab rather than quoted verbatim.

4. The Prompt Chain

Paste these prompts in order into a fresh chat session. Phase A must be sent in full before Phase B.

Phase A — Configuration (No Code Generation)

Prompt 1 — Execution Policy

```
We are entering CONFIGURATION MODE.
Execution State = LOCKED

Do NOT generate: Java, XML, YAML, SQL, Tests, pom.xml, application.properties.

During this phase: store all instructions, do not execute implementation.

Reply only: "Configuration accepted. Waiting for the next instruction."

Implementation begins only after I send: EXECUTION STATE = UNLOCKED
```

Prompt 2 — System Persona

```
You are a Principal Java Architect with 25+ years of experience.
Expert in: Java 21, Spring Boot, Microservices, DDD, Security, Performance, Testing, Design Patterns.
Generate enterprise-quality solutions.
```

Prompt 3 — User Persona

```
I am a Senior Java Developer. Assume I understand Java, Spring Boot, REST, Maven.
Do not explain beginner concepts. Provide concise architectural guidance.
```

Prompt 4 — Context Manager

```
Project: User Registration Service
Technology: Java 21, Spring Boot 3.5, Maven
Future: JWT, Kafka, Docker, Kubernetes
Architecture: Layered Architecture
```

Prompt 5 — Question Refinement

```
Before executing any future request: refine my prompt, remove ambiguity,
improve technical accuracy, preserve intent. Return only the refined prompt.
Do NOT generate implementation.
```

Prompt 6 — Meta Language (updated)

```
Entity = JPA Entity
Repository = Spring Data Repository
DTO = Request/Response DTO
Service = Business Layer (methods annotated @Transactional where they mutate state)
Controller = REST Controller
Validation = Jakarta Validation
Exception = Custom exception classes, thrown from the Service layer
Exception Handler = Global @ControllerAdvice handler, generated separately from Exceptions
PasswordPolicy = Passwords are hashed with BCryptPasswordEncoder before persistence, never
stored or returned in plain text
```

Prompt 7 — Few-shot Examples

```
Input: Generate Repository → Output: Repository only
Input: Generate Entity → Output: Entity only
Input: Generate Controller → Output: Controller only (reuse Service)
```

- If the user asks for a Repository, don't generate Entity, Service, Controller, etc.

Prompt 8 — Output Template

```
Purpose
Design Decisions (2–4 sentences of rationale — do not omit even under Prompt 10)
Implementation Summary
Files Generated
Possible Improvements
Next Step
```

Prompt 9 — Output Automator

```
Automatically include: Architecture Summary, Files Generated, Design Decisions, Next Logical
Task.
```

Prompt 10 — Reasoning Scope

```
Do not display step-by-step internal reasoning or chain-of-thought.
Do include a brief final rationale in the "Design Decisions" section of the Output Template.
```

Prompt 11 — Fast Checklist (pre-generation)

```
Before every implementation verify: Java 21, Spring Boot 3.5, SOLID, REST Standards,
Validation, Exception Handling, Transaction Boundaries, Password Hashing Policy, No duplicate
code.
```

Prompt 12 — Reflection (post-generation)

```
After every implementation review: Security, Performance, Validation,
Layer violations, Duplicate logic, Transaction correctness.
```

Prompt 13 — Alternative Approaches

```
Only when I explicitly ask for an architecture comparison:
Compare Layered, Hexagonal, and Clean Architecture, and recommend the best fit.
Do not run this comparison automatically during level generation.
```

Prompt 14 — Flipped Interaction

```
Ask questions only if absolutely necessary. Otherwise proceed with sensible assumptions.
```

Prompt 15 — Level Plan

```
Treat development as levels, unlocked one at a time, confirmed by me before continuing:
Level 1  Bootstrap (pom.xml)
Level 2  Configuration (application.properties)
Level 3  Package structure
Level 4  User entity
Level 5  Repository
Level 6  DTOs
Level 7  Custom Exceptions
Level 8  Service
Level 9  Controller
Level 10 Validation refinement + Global Exception Handler
Level 11 Testing
```

Custom Exceptions are placed before Service, since Service methods throw them. The Global Exception Handler is placed later, since it needs to know every exception type before it can be wired up.

Prompt 16 — Continuation Gate

```
After each completed level, ask "Continue to next level?" and wait for my confirmation.
Generate only the next logical level. Never regenerate previous levels.
```

Phase B — Implementation

Prompt 17 — Unlock Implementation

```
EXECUTION STATE = UNLOCKED
Begin implementation. Generate one level at a time. Wait for confirmation before continuing.
```

Level 1 — Maven pom.xml

Dependencies: Spring Web, Validation, Spring Data JPA, DevTools, Actuator, H2, Lombok, Spring Boot Test.

Level 2 — application.properties

Level 3 — Package structure

Level 4 — User entity

Fields: id, name, email, password, phone, createdAt, updatedAt.

Apply the PasswordPolicy from the meta-language glossary — the password field must never be exposed via toString(), equals()/hashCode(), or the Response DTO.

Level 5 — UserRepository

Support: findByEmail(), existsByEmail().

Level 6 — UserRequest and UserResponse DTOs

UserResponse must exclude the password field entirely.

Level 7 — Custom Exceptions

Define exception types the Service layer will throw, e.g. duplicate-email and not-found cases. Do not generate the @ControllerAdvice handler yet — that comes in Level 10.

Level 8 — UserService

Operations: Register, Update, Delete, Get By Id, Get All.

- Hash passwords with BCryptPasswordEncoder before persistence.
- Use existsByEmail() to reject duplicate registrations (idempotency for the future Kafka-triggered registration flow).
- Apply @Transactional to state-mutating methods per the meta-language convention.
- Throw the custom exceptions defined in Level 7, not generic exceptions.

Level 9 — UserController

Use ResponseEntity. Return proper HTTP status codes.

Level 10 — Validation Refinement + GlobalExceptionHandler

Wire the Level 7 custom exceptions into a single @ControllerAdvice, plus Jakarta Validation error handling.

Level 11 — Testing

JUnit tests, Mockito tests, Controller tests, Service tests.

Final Verification

Verify: Compilation, Imports, Packages, Validation, Security, Performance. Suggest improvements.

5. Notes for Students

5.1 Why phase-locking matters

Separating “configuration” from “implementation” (Prompt 1 / Prompt 17) prevents a common failure mode: the model jumping ahead and generating code before all constraints are in place. Once EXECUTION STATE = UNLOCKED is sent, every earlier prompt is still active context — it isn't a one-time instruction, it's a standing constraint.

5.2 Why ordering dependencies matters in level-gating

A level plan is only as good as its dependency graph. Before locking a level sequence, ask: “what does this level need that doesn't exist yet?” Custom Exceptions in this lab are a concrete example — Service code that throws an exception type needs that type defined first, even though “Exception Handling” intuitively feels like a late-stage concern. That's why Custom Exceptions (Level 7) sit before Service (Level 8), while the Global Exception Handler, which needs to know every exception type, waits until Level 10.

5.3 Watch for instruction conflicts, not just instruction gaps

Most prompt-engineering advice focuses on what's missing from a prompt chain. Equally important is checking whether two active instructions contradict each other under certain conditions — as Reasoning Scope and the Template pattern did here. These conflicts often only surface a few levels in, once both instructions are simultaneously “hot.”

5.4 Security and non-functional requirements degrade silently

Storing a raw password field would compile and run just fine — nothing about it looks like an error. Non-functional requirements (hashing, transaction boundaries, idempotency) need to be stated explicitly in the meta-language or checklist layer, because the model will not infer them just from field names like “password.”

5.5 Scope broad instructions, don't just add them

Prompt 13 (Alternative Approaches) is useful, but only when the student actually wants an architecture debate. An instruction with no trigger condition runs on every turn by default — when adding a broad pattern like this, pair it with an explicit "only when" clause.

5.6 Suggested exercise

1. Run the chain as written through Level 4, then read the generated User entity and check whether the password field is actually excluded from equals()/hashCode()/toString().
2. Deliberately remove Prompt 11 (Fast Checklist) and rerun Level 8 — compare whether @Transactional and BCryptPasswordEncoder still appear without being explicitly checked for.
3. Try swapping the order of Level 7 (Custom Exceptions) and Level 8 (Service) and observe whether the model produces a Service that references an exception class that doesn't exist yet.

5.7 Reusing this pattern set on a different project

Everything in Section 2 is stack-agnostic. To reuse this lab for a different service (e.g. a Node/Express or .NET project), keep Phase A's structure and only change: the System Persona's tech list (Prompt 2), the Context Manager's tech stack (Prompt 4), and the Meta Language's framework-specific terms (Prompt 6). The Level plan in Prompt 15 should be re-derived from that project's own dependency graph rather than copied as-is.